

## ✓ 1. Judul Praktikum

Praktikum Struktur Data - Modul 3: Konsep Struktur Data, ADT, dan Analisis Kompleksitas Algoritma.

## 2. Tujuan Praktikum

1. Memahami konsep dasar struktur data.
2. Menjelaskan pengertian ADT (Abstract Data Type).
3. Memahami pentingnya efisiensi algoritma dan mengenali kompleksitas waktu.
4. Menggunakan Big-O Notation sederhana.
5. Membandingkan efisiensi beberapa algoritma dasar.

## 3. Dasar Teori

- **Struktur Data:** Cara menyimpan, mengorganisasikan, dan mengelola data agar dapat digunakan secara efisien. Struktur data terbagi menjadi Linear (berurutan seperti Array, List, Stack) dan Non-Linear (hierarki/jaringan seperti Tree, Graph). Pemilihan yang tepat mempengaruhi kecepatan, penggunaan memori, dan kemudahan pengembangan.
- **Abstract Data Type (ADT):** Model konseptual yang mendefinisikan jenis data, operasi, dan perilaku tanpa berfokus pada implementasi detail. Contohnya ADT Stack dengan operasi Push, Pop, dan Peek.
- **Kompleksitas & Big-O Notation:** Digunakan untuk menggambarkan pertumbuhan waktu eksekusi algoritma terhadap jumlah data. Contoh kompleksitas umum adalah Konstan  $O(1)$ , Linear  $O(n)$ , dan Kuadratik  $O(n^2)$ . Algoritma efisien sangat penting untuk menangani data besar.

## 4. Langkah Praktikum

Berikut adalah implementasi Prosedur Praktikum (6.1 - 6.5) dan beberapa Studi Kasus (7.1 - 7.4) dari modul.

```

1 # --- 6. PROSEDUR PRAKTIKUM ---
2 print("--- 6.1 Struktur Data Dasar ---")
3 data_dasar = [10, 20, 30, 40]
4 print(f"Data: {data_dasar}")
5 print(f"Elemen index ke-0: {data_dasar[0]}\n")
6
7 print("--- 6.2 Abstract Data Type (ADT) Stack ---")
8 stack = []
9 stack.append(10) # Push 10
10 stack.append(20) # Push 20
11 print(f"Mengambil data (Pop): {stack.pop()}\n")
12
13 print("--- 6.3 Analisis Kompleksitas Linear O(n) ---")
14 data_linear = [1, 2, 3, 4, 5]
15 for i in data_linear:
16     print(i, end=" ")
17 print("\n\n")
18
19 print("--- 6.4 Analisis Kompleksitas Kuadratik O(n^2) ---")
20 for i in range(3):
21     for j in range(3):
22         print(f"({i},{j})", end=" ")
23 print("\n\n")
24
25 print("--- 6.5 Perbandingan Algoritma (Pencarian) ---")
26 data_cari = [10, 20, 30, 40]
27 target = 30
28 for i in data_cari:
29     if i == target:
30         print(f"Target {target} ditemukan dalam iterasi O(n)\n")

```

```

31
32
33 # --- 7. STUDI KASUS ---
34 print("--- SK 1: Penyimpanan Data Mahasiswa ---")
35 data_mahasiswa = ["Andi", "Budi", "Citra"]
36 print(f"Daftar Mahasiswa: {data_mahasiswa}\n")
37
38 print("--- SK 2: Simulasi Stack Buku ---")
39 tumpukan_buku = []
40 tumpukan_buku.append("Buku A") # Push
41 tumpukan_buku.append("Buku B") # Push
42 tumpukan_buku.append("Buku C") # Push
43 print(f"Tumpukan awal: {tumpukan_buku}")
44 buku_diambil = tumpukan_buku.pop() # Pop
45 print(f"Buku yang diambil paling atas: {buku_diambil}")
46 print(f"Sisa tumpukan: {tumpukan_buku}\n")
47
48 print("--- SK 3: Analisis Perulangan ---")
49 print("Single loop mengeksekusi proses sebanding jumlah n.")
50 print("Nested loop mengeksekusi proses sebanding kuadrat n (n*n).\n")
51
52 print("--- SK 4: Pencarian Data (ID Koperasi) ---")
53 id_koperasi = [101, 102, 103, 104, 105]
54 cari_id = 104
55 ketemu = False
56 for no_id in id_koperasi:
57     if no_id == cari_id:
58         ketemu = True
59         break
60 print(f"ID {cari_id} {'Ditemukan' if ketemu else 'Tidak Ditemukan'}")

```

--- 6.1 Struktur Data Dasar ---

Data: [10, 20, 30, 40]

Elemen index ke-0: 10

--- 6.2 Abstract Data Type (ADT) Stack ---

Mengambil data (Pop): 20

--- 6.3 Analisis Kompleksitas Linear  $O(n)$  ---

1 2 3 4 5

--- 6.4 Analisis Kompleksitas Kuadratik  $O(n^2)$  ---

(0,0) (0,1) (0,2) (1,0) (1,1) (1,2) (2,0) (2,1) (2,2)

--- 6.5 Perbandingan Algoritma (Pencarian) ---

Target 30 ditemukan dalam iterasi  $O(n)$

--- SK 1: Penyimpanan Data Mahasiswa ---

Daftar Mahasiswa: ['Andi', 'Budi', 'Citra']

--- SK 2: Simulasi Stack Buku ---

Tumpukan awal: ['Buku A', 'Buku B', 'Buku C']

Buku yang diambil paling atas: Buku C

Sisa tumpukan: ['Buku A', 'Buku B']

--- SK 3: Analisis Perulangan ---

Single loop mengeksekusi proses sebanding jumlah n.

Nested loop mengeksekusi proses sebanding kuadrat n ( $n*n$ ).

--- SK 4: Pencarian Data (ID Koperasi) ---

ID 104 Ditemukan

## 5. Analisis

- **Struktur Data Linear:** Implementasi list pada Python secara alami mendukung struktur data linear di mana elemen diakses menggunakan indeks secara konstan  $O(1)$ .

- **ADT Stack:** ADT Stack mematuhi prinsip LIFO (Last In First Out). Fungsi `append()` bertindak sebagai operasi *Push*, dan `pop()` untuk menghapus sekaligus mengembalikan elemen terakhir.
  - **Kompleksitas:** Pada nested loop, jika perulangan luar berjalan 3 kali dan dalam berjalan 3 kali, total iterasi adalah  $3 \times 3 = 9$ . Ini membuktikan pertumbuhan kuadratik  $O(n^2)$ , yang akan sangat lambat jika  $n$  bernilai ribuan.
- 

## 6. Kesimpulan

Pemilihan struktur data (seperti list untuk array atau stack) secara langsung memengaruhi kemudahan operasi. Pemahaman akan efisiensi (Big-O) esensial untuk memprediksi perilaku program saat data membesar. Pencarian dengan loop tunggal menghasilkan kompleksitas  $O(n)$  yang masih wajar untuk ukuran data standar.